Entropy Aware Training for Fast and Accurate Distributed GNN

Dhruv Deshmukh*, Gagan Raj Gupta†, Manisha Chawla‡, Vishwesh Jatala§ and Anirban Haldar¶
Department of CSE, IIT Bhilai, India
Email: {*dhruvr, †gagan, ‡manishach, §vishwesh, ¶anirbanh}@iitbhilai.ac.in

2023 IEEE International Conference on Data Mining (ICDM) | 979-8-3503-0788-7/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICDM58522.2023.00112

***Abstract*— Several distributed frameworks have been developed to scale Graph Neural Networks (GNNs) on billion-size graphs. On several benchmarks, we observe that the graph partitions generated by these frameworks have heterogeneous data distributions and class imbalance, affecting convergence, and resulting in lower performance than centralized implementations. We holistically address these challenges and develop techniques that reduce training time and improve accuracy. We develop an Edge-Weighted partitioning technique to improve the micro average F1 score (accuracy) by minimizing the total entropy. Furthermore, we add an asynchronous personalization phase that adapts each compute-host's model to its local data distribution. We design a class-balanced sampler that considerably speeds up convergence. We implemented our algorithms on the DistDGL framework and observed that our training techniques scale much better than the existing training approach. We achieved a (2-3x) speedup in training time and 4% improvement on average in micro-F1 scores on 5 large graph benchmarks compared to the standard baselines.**

***Index Terms*—Distributed ML, graph neural networks, class imbalance**

I. Introduction

Graph neural networks (GNNs) have made tremendous progress in recent years and have achieved state-of-the-art performance in diverse applications [1], including social network analysis, credit-card fraud detection [2], recommender systems, etc. A core operation in GNNs is message-passing to aggregate information from neighbors, which is then used to learn task-specific vertex embeddings. These operations require internal graph structures to be stored (in memory) during forward and backward propagation, making it very difficult to scale on industrial-grade graphs with billion-scale edges. Several distributed GNN frameworks [3], [4] have been developed recently to address this problem.

One of the critical steps in most of the above frameworks [3] to achieve scalability is partitioning the input graph into disjoint sub-graphs of similar size and assigning one sub-graph (partition) to every compute host in the cluster for training. During the distributed training process, in every iteration, a sample (batch) of labeled training nodes is used by every compute host to estimate the local gradients, using which a global average is computed and used to update the global model. The final model obtained at the end of training performs tasks such as label prediction of the test vertices present at each compute host in parallel.

Similar to [5], we observe in Fig. 1a), that the entropy[1] of these partitions has significant variation due to non-i.i.d labels. This affects convergence results in lower performance than centralized implementations, especially when number of compute hosts is increased. Fig. 1a) shows the micro average F1 scores on the test set per compute node after distributed training on the OGBN-Products dataset with 16 compute hosts. We observe that the compute hosts having partitions with lower entropies typically achieve higher accuracy (same as micro avg. F1 score), although there are some outliers due to complex factors involved. Fig. 1b) shows the huge imbalance in the label distribution in the OGBN-Products dataset, which leads to a bias in the models toward majority classes. Graph partitioning can worsen this further.

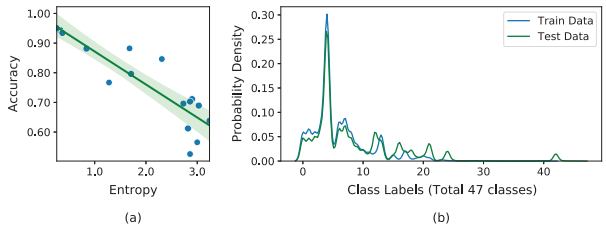

Fig. 1: Challenges in Distributed GNN Training. a) Variation in the entropy of partitions affects accuracy in OGBN-Products dataset with 16 partitions. A linear regression line with 95% confidence interval is overlayed. b) Class imbalance and out-of-distribution in OGBN-Products.

We holistically address these challenges and develop techniques that reduce training time and improve accuracy. We extensively study the impact of the entropy of partitions on the performance of GNNs. We design an efficient edge-weighted partitioning algorithm that aims to minimize partitions' total entropy to improve micro average F1 scores. In this algorithm, the weight of the edge is assigned based on the node degree and similarity of features of the corresponding adjacent nodes. Since nodes with similar labels often have similar features, their edge weights are higher. Edge-weighted partitioning schemes aim to minimize the sum of edge weights of cut edges, resulting in partitions with nodes of similar labels and thus reduce total entropy.

We then investigate personalization [6], [7] of models at each compute node in an asynchronous mode to adapt to the local data distributions. This results in local personalized models at each compute node, significantly improving performance. At the same time, this helps in faster training

[1]Entropy = $-\Sigma_i p_i log(p_i)$ where p_i is the fraction of the i-th label

979-8-3503-0788-7/23/$31.00 ©2023 IEEE
DOI 10.1109/ICDM58522.2023.00112

as the communication and synchronization costs of averaging gradients across all partitions are reduced greatly.

Finally, we develop a class-balanced sampler (CBS), that promotes equitable representation of minority classes in every batch during training. For large graphs, our sampler reduces the number of training examples (from the majority classes) per epoch, resulting in a faster completion time per epoch. We implement these techniques in the DistDGL framework [3] and perform extensive experiments on large-scale graph benchmarks. We briefly summarize the contributions of our paper:

- Development of novel entropy-aware partitioning algorithms to minimize the total entropy
- Development of a new class-balanced sampler to address the class-imbalance problem in real-world graphs and speed-up training.
- Asynchronous personalization of graph models to local data distributions to achieve higher performance.
- Extensive experimental evaluation on large-graph benchmarks including OGBNPapers-100M with a billion edges on commodity HPC clusters. Our techniques achieve a speedup of 2-3x, improve micro-F1 score by 4% and on average in 5 large graph benchmarks compared to the standard DistDGL implementation.

The rest of the paper is organized as follows. We provide the relevant background for distributed GNN and related research in Section II. This is followed by a detailed description of our algorithms for entropy aware distributed GNN training in Section III. We then describe the datasets and experimental setup in Section IV. The experimental results and ablation studies are discussed in Section V followed by the conclusions.

II. Background and Related Work

Graph neural networks learn representations of vertices/edges by aggregating the messages received from the neighborhood vertices. Consider a graph $G(V, E)$ having V vertices, E edges, L Labels. Each vertex (or edge) in the graph is associated with a feature vector, $\mathbf{x_v}$, which is used to initialize $\mathbf{h_v}^0$. For a GNN consisting of k layers ($k > 0$), the embedding $\mathbf{h}_v^i$ of v at the i^{th} layer is computed from the embeddings generated by the previous layer by applying an aggregation function AGG_i and a non-linear function σ_i on the messages received from the neighborhood of v (denoted as $N(v)$), its own feature after multiplying with the weight matrix $\mathbf{W}^i$. The equations to compute $\mathbf{h}^i$ from $\mathbf{h}^{i-1}$ are listed below.

$$\mathbf{h}^i_{N(v)} = AGG_i(\mathbf{h}^{i-1}_u, \forall u \in N(v)) \tag{1}$$

$$\mathbf{h}^i_v = \sigma_i(\mathbf{W}^i.CONCAT(\mathbf{h}^i_{N(v)}, \mathbf{h}^{i-1}_v)) \tag{2}$$

The final embeddings, $\mathbf{h}^k_v$, are used to make the predictions, and the gradients of the loss w.r.t. $\mathbf{W}^i$ are used to update $\mathbf{W}^i$ till convergence is reached.

Many GNN models were developed to improve accuracy; these models differ in the way the aggregations are performed. GraphSAGE [8] aggregates neighborhood vertices features along with its own features. It has been widely used for distributed settings as it uses neighborhood sampling techniques to reduce computation and communication overhead. It has been observed that GraphSAGE achieves reasonable performance when trained with 2-layer GNN, and adding further layers causes over-fitting and increases communication overhead [9], [10]. In this paper, we have used the GraphSAGE algorithm in all the experiments.

Distributed GNN Frameworks: DistDGL [3], PipeGCN [11], and Adgraph [12] adopt METIS graph partitioning. P3 [4] uses random hash, and DistGNN [13] uses vertex cut partition to partition the graph. However, these frameworks do not consider class imbalance.

DistDGL is based on distributed PyTorch and has configurable functions for partitioning, sampling, etc., which we use to implement our algorithms. Our algorithms proposed in this paper focus on improving various performance metrics while reducing training time in the distributed setting.

Graph Partitioning: Distributed graph analytics, or GNN requires partitioning the input graph among multiple hosts. Graph partitioning has been studied for decades [14]–[16]. METIS [16] has been widely used to partition a graph in distributed GNN models [3], [17]. It minimizes the number of edge cuts among the partitions and balances the nodes in each partition. Most partition techniques achieve two primary goals: (1) balance computation load among the hosts and (2) reduce communication overhead among the hosts. To the best of our knowledge, none of the partitioning techniques have considered minimization or balancing entropy to improve distributed GNNs' performance.

Graph Sampling: It has been observed that GNN-based algorithms have poor performance when the label distribution is skewed. To address this problem, [2], [18], [19] were developed. GraphSMOTE [18] attempts to extend the classical approach of synthetic minority oversampling algorithms to GNNs by sampling in the representation space and using an edge generator. PC-GNN [2] uses a label-balanced sampler to construct sub-graphs and neighbors for message aggregation during training for binary classification problems. We generalize these techniques to multi-class node classification and distributed GNN training.

Personalization: Personalization or fine-tuning of models is an important step in dealing with the heterogeneity of clients. This has been studied recently in the Federated learning literature [6], [7], [20], [21] but hasn't been evaluated thoroughly in the context of distributed GNNs. Personalization improves accuracy when the test data distribution matches with the train data distribution. Ada-GNN [7] used a basic form of personalization in a centralized implementation.

Performance Metrics: Micro-F1 computes accuracy using all testing examples at once, while Weighted Macro-F1 (Weighted-F1) is computed by giving a weight proportional to the class frequency during the averaging of individual F1 scores. The effectiveness of the distributed GNN is also measured using two other metrics: training time and epoch time. Training time denotes the maximum training time across the compute hosts. Epoch time denotes the time taken for one

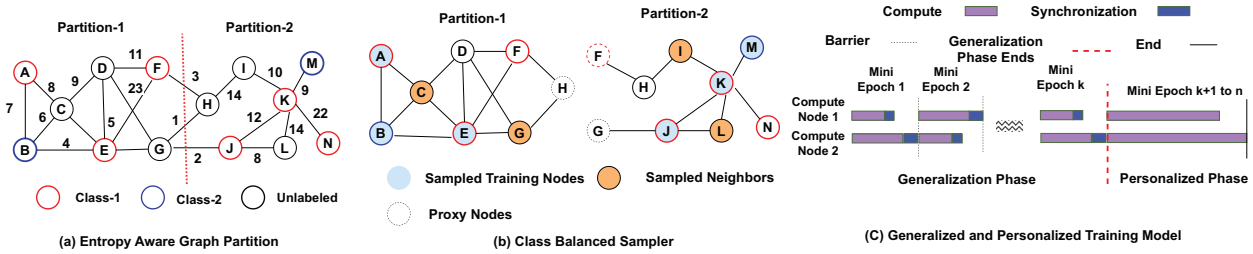

Fig. 2: Overview of the proposed approaches for high-performance distributed GNN.

complete pass through the entire training dataset.

III. Our Algorithms

Figure 2 gives an overview of the proposed approaches. First, we construct a weighted graph from the unweighted graph, where the weight of an incoming edge is determined by the node degree and feature similarity of the adjacent nodes. Then, we partition the weighted graph among the compute hosts using the weighted METIS [16], as shown in Figure 2(a).

Once the graph is partitioned, we use a class-balanced sampler (CBS) to select a subset of training nodes during a mini-epoch, as shown in Figure 2(b). CBS samples training nodes of the minority classes with higher probability and nodes of majority classes with lower probability. Once the mini-epoch training nodes are chosen, a random training batch is created during each iteration. K neighbors are sampled for each node in this batch to perform message aggregation using the update Equation (1).

The distributed training occurs in two phases: generalization followed by personalization, as shown in Figure 2(c). The training starts with the generalization phase (phase-0), which aims to learn a global model that performs well on the entire data distribution. The personalization phase (phase-1) is started in asynchronous mode when the loss curve of the generalization phase starts to flatten. Training stops on each local node once local convergence is reached and the test performance metrics are evaluated.

Section III-A, III-B, and III-C discuss the details of our partitioning, sampling, and personalized training algorithms, respectively.

A. Entropy-Aware Graph Partitioning

We aim to create an effective partitioning algorithm that minimizes the total entropy of the partitions. This is achieved by assigning weights to the edges based on two factors: (1) the similarity of their features and (2) the degree of the neighborhood vertex. The weighted partitioning scheme ensures that (1) the nodes that have similar features (and labels) have a higher probability of being in the same partition; this is expected to help in reducing average entropy and improve Micro-F1 scores (2) for nodes having a lower degree, their neighbors have a high probability of being in the same partition, which can help in reducing the communication overhead [22] for distributed GNNs and improve scalability.

Consider u and v are any two vertices of the graph, and $|N(v)|$ denotes the size of the neighborhood of v. Algorithm 1 shows the edge-weight assignment process. We compute the similarity of two neighborhood vertices as the dot product of their initial features. Next, we compute an approximate probability that a neighborhood vertex u of v is among the K nodes sampled for message aggregation by the GraphSage algorithm [8], [22]. If u has a low degree, the value of p will be closer to 1. We use a weighted combination of similarity and p as the edge weight, as shown in Line 5. c is a hyper-parameter that depends on the properties of the graph. Finally, we use the weighted-METIS algorithm [16], a multilevel recursive-bisection-based partitioning technique, to partition the newly constructed weighted graph with roughly the same number of vertices in each partition.

Algorithm 1: Edge Weighted Graph Partitioning

Input : Graph G
Output: Graph Partitions $(parts)$
1 **for** $v \in V$ **do**
2 **for** $\forall u : (u, v) \in E$ **do**
3 $similarity = \mathbf{h}_u^0 \cdot \mathbf{h}_v^0$
4 p = 1-e$^{\frac{-K}{|N(v)|}}$
5 W_{uv} = $(c * similarity + p) * 100$
6 **end**
7 **end**
8 parts = WeightedMETIS(G, W)
9 Save parts

The time complexity of the above Algorithm 1 is $O(|E||D|)$, where $|E|$ is the number of edges in the graph as each edge is traversed exactly once, and similarity can be computed in $O(D)$ where D is the feature dimensionality.

B. CBS: Class Balanced Sampler

Our sampler, CBS, is inspired by the pick sampler [2] which assigns the sampling probability of a training node based on its normalized degree and class label frequency. Instead of using all the training nodes in an epoch, CBS generates a random subset using the following probability per training node.

$$P(v) = \frac{||\hat{A}(:, v)||^2}{CF(class[v])} \tag{3}$$

where $\hat{A} = D^{-\frac{1}{2}} A D^{\frac{1}{2}}$ is the normalized adjacency matrix and D is the degree matrix. The size of the subset is a fraction, typically 25% of the training nodes. A complete training pass on this subset is called a mini-epoch. In every mini-epoch, a new subset is sampled from the entire training set of that

compute host. Each mini-epoch is further divided into iterations where a random batch is sampled every iteration. This increases the probability that the training nodes of minority classes will participate in every batch. An additional benefit of this approach is that the mini-epochs run much faster and have better convergence, explained in Section V.

C. Personalization under Class Imbalance

A global model is trained in the generalization phase (phase-0) till reasonable accuracy is achieved. The models on each compute host are initialized with the same parameters and are kept in sync by applying the same update equation. In the personalization phase (phase-1), the aggregation is stopped, and each node learns independently to tune each local model according to its local training data. This reduces synchronization overheads and speeds up the training process considerably. A parameter controls the proportion of generalization and personalization in a given number of epochs.

During phase-1, to control model overfitting to local distributions, a regularization loss term has been added to the Loss function in Eq. 4 to keep the personal model weights close to the general model learned after phase-0. The squared norm of the difference between these tensors is added to the loss function as a regularization term. Another form of regularization that has been added is early stopping. In phase-0, the early stopping happens based on the average micro-F1 score on the validation set of all partitions; all compute hosts stop simultaneously. In phase-1, the individual micro-F1 score decides when to stop training. The best model is saved. The early stopping of both phases is done independently of the other phase.

Thus, the final loss function of the i^{th} compute host during phase-1, including the regularization term, is:

$$Loss_i = \sum_{j=1}^{n} CrossEntropy_j(\mathbf{W_i}) + \lambda||\mathbf{W}_i^P - \mathbf{W}^G||_2 \quad (4)$$

Here n is the number of training examples in a batch, $\mathbf{W}_i^P$ are the personal weights of i-th computer host, and $\mathbf{W}^G$ are the general weights same for all the compute hosts.

IV. Experimental Setup

We implemented[2] the proposed strategies using DGL 0.9 [23]. We use PYMETIS, a python wrapper for METIS, for performing the weighted partitions. We evaluated our approach on several datasets using GraphSAGE [8]. Table I shows the list of datasets [3] used for our evaluation.

We evaluated our algorithms using two platforms. For larger datasets, Reddit, OGB-Products, and OGB-Papers, we conducted experiments on a shared commodity cluster using up to 16 compute hosts. For Flickr and Yelp, we use a dedicated Amazon AWS EC2 cloud cluster with 4 VMs. Each VM is equipped with (R6a.xlarge) Intel(R) Xeon(R) Es-2686 v4 CPU having 8 processing cores with 2.30 GHz frequency, 32GB RAM, and 200 GB external storage. For EW partitioning of larger graphs like Reddit and OGBN-Products VM equipped with (R6a.4xlarge) having 128 GB RAM has been used.

For all our experiments, we choose the learning rate as 0.001, the neighborhood sample as (25,25), and the number of layers as 2, unless specified. Restricting to two layers limits the bottleneck of communication time and speeds up distributed training. The results reported are an average of over five runs. For Flickr, we don't use the sampler as this drastically reduces the number of nodes in an epoch. For Yelp, the convergence happens slowly with the sampler, and increasing the learning rate makes the curve noisier; hence, the weighted scheme training has been run for more epochs, while in others, even the 100 epochs with sampling are enough.

V. Results and Discussions

This section describes the experimental analysis of our proposed ideas. We use the following notations to describe the various approaches evaluated in the paper.

- **METIS:** Default METIS graph partitioning scheme used by the DistDGL
- **DistDGL:** Default distributed training followed by DistDGL using METIS partition and Graph Sage
- **EW:** Edge weighted partitioning scheme
- **CBS:** Class balanced sampling scheme
- **GP:** Two-phase (Generalize and Personalize) distributed training of the model

A. Comparing overall performance

Table II shows the performance comparison of our EW partitioning combined with GP and CBS with the baseline (**DistDGL**) on 4 compute hosts. We apply the distributed training with generalization and early stopping for the baseline. The table reports micro-F1, weighted-F1 scores, training time, and highlights (in blue) the best-performing score for each dataset and metric. From the results, we observe that our proposed algorithms achieve an average improvement of 4% on micro-F1 scores and 1.9% on weighted-F1 scores. This is because **EW** yields a few partitions with very low entropy and obtains high and moderate scores on the remaining partitions to achieve the best overall accuracy. On OGBN-Papers[3], there is a relative improvement of 16.2% on the micro-F1 score and 11.3% on the weighted-F1 score by performing **GP** with class balancing.

Table II also reports the training time for each training technique. It is evident that **EW+GP+CBS** converges faster than the **DistDGL** and shows a significant reduction in training time in large graph benchmarks: Reddit by 2.2x, OGBN Products by 3.4x and OGBN Papers by 1.8x). These reductions indicate the usefulness of CBS and the personalization technique. This can be understood further with the help of Figure 3 where we plot the validation micro-F1 and training loss for various partitioning algorithms. We trigger the personalization phase

[2]Code available at https://github.com/Anirban600/EAT-DistGNN

[3]This experiment was repeated twice on 16 compute hosts using METIS partitions in both cases.

Data Set	Nodes	Edges	Features	Labels	Train/Val/Test %	Avg. Degree	Comments
Flickr	89,250	899,756	500	7	50/25/25	20	Noisy Labels
Yelp	716,847	13,954,819	300	100	75/15/10	39	Multilabel
Reddit	232,965	114,615,892	602	41	66/10/24	492	High Node degree, Feature Dimensions
OGBN-Products	2,449,029	61,859,140	100	47	8/2/90	51	Out of Distribution
OGBN-Papers	111,059,956	1,615,685,872	128	172	78/8/14	29	∼98% Unlabelled

TABLE I: Statistics of Graph Datasets spanning a wide range of applications used in this paper for experimental evaluation.

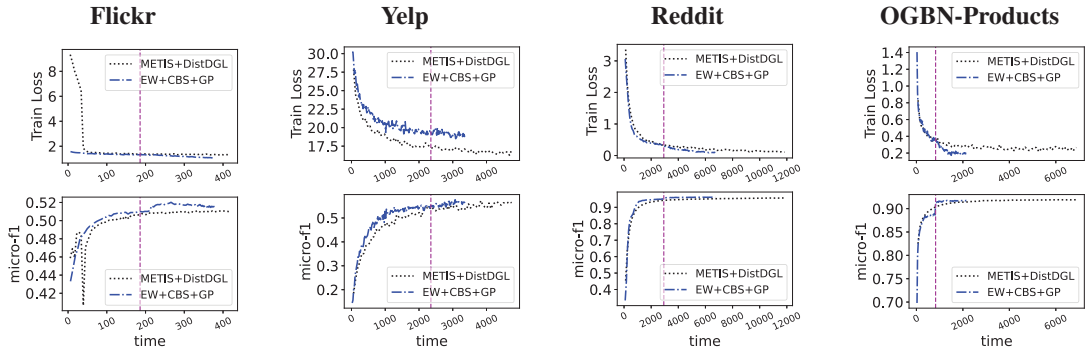

Fig. 3: The convergence curves for training loss, validation micro scores for Flickr, Reddit, Yelp and OGBN-Products using various partitioning schemes. The magenta vertical line in all plots represents the time where personalization starts.

	F1 Metric	DistDGL	EW+CBS+GP
Flickr	Micro	51.43±0.18	51.98±0.18
	Weighted	43.08±0.36	43.64±0.35
	Train Time (s)	477 (1.15×)	414
Yelp	Micro	58.67±1.42	58.5 ± 1.36
	Weighted	54.88±1.94	54.55 ± 1.8
	Train Time (s)	4971(1.35×)	3670
Reddit	Micro	94.87±0.11	96.17±0.16
	Weighted	95.5 ± 0.11	96.07±0.13
	Train Time (s)	12552 (2.2×)	5724
OGBN Products	Micro	78.36±0.35	78.94±0.32
	Weighted	77.08±0.42	77.94±0.25
	Train Time (s)	7194 (3.4×)	2115
OGB Papers	Micro	43.9	**51.02**
	Weighted	44.23	**49.25**
	Train Time (s)	12763 (1.8×)	6996

TABLE II: Comparing performance metrics of various algorithms. Scores are reported as percentages and best performing technique is highlighted.

when the loss curve starts to flatten. For **EW+GP+CBS**, Loss includes an extra term due to regularization.

Results show that **EW+GP+CBS** converges faster than **DistDGL** and there is a noticeable jump in micro-F1 scores for OGBN-Products and Flickr as soon as we begin the personalization phase. We implement distributed early stopping to minimize the epochs wasted and save the best model obtained. In the case of Yelp, convergence is noisy and slow, so even though there is no jump in the score, we obtain a reduction in training time. Finally, there is a sharper decay in all the loss curves as soon as the personalization starts, which indicates faster convergence. This demonstrates the effectiveness of personalization in adapting to the local distribution of data.

*B. Scalability with **EW+GP+CBS***

Metric	Partitions	DistDGL	EW+CBS+GP
Training Time (s)	4	7194 (3.8×)	1876
	8	3632 (3.2×)	1113
	16	2261 (2.8×)	801
Epoch Time (s)	4	37.84	12.54
	8	18.92	6.76
	16	11.55	4.06
Micro F1	4	78.36	79.48
	8	77.2	79.71
	16	75.3	76.4

TABLE III: Scaling results for OGBN Products

We perform scalability experiments with OGBN Products dataset for 4, 8, and 16 compute hosts and present the results in Table III. We notice that the micro-F1 scores start degrading for **DistDGL** as we increase the number of hosts because of the increased heterogeneity of partitions. In contrast, for **EW+GP+CBS**, the micro-F1 score is best for the 8 partitions and is always better than **DistDGL** for all settings. CBS reduces the Epoch time by a factor of 3 as compared to the **DistDGL**. **EW+GP+CBS** achieves a consistent 3x reduction in training time for all settings.

C. Comparison with the centralized model

In Table IV, we compare the performance metrics obtained by distributed training of GNN (**DistDGL and EW+GP+CBS**) with the state-of-the-art centralized training [9] of GraphSAGE using 2-layers. The results show that the **DistDGL** degrades performance by 1-2%

Method	Flickr	Reddit	OGBN-Products
Centralized	**52.26**	**96.34**	78.20
DistDGL	51.43	94.47	78.36
EW+GP+CBS	51.98	96.17	**79.71**

TABLE IV: Comparison with centralized GraphSAGE model

D. Discussion

We now present our analysis of the reasons behind the improvements obtained by our techniques and when to use each of them.

Partition Method	Reddit		Yelp		OGB-Products	
	$\mathbf{H}(\mathcal{P})$	time	$\mathbf{H}(\mathcal{P})$	time	$\mathbf{H}(\mathcal{P})$	time
METIS	3.35	31.94	38.05	4.8	2.80	42.1
EW	3.21	172.9	37.92	91.2	2.44	784.7

TABLE V: Average entropy and time to partition (in sec), across various graph partitioning algorithms.

Table V shows entropies of various partitioning algorithms. We note that EW consistently reduces the total entropy of partitions as compared to METIS and improves the micro F1 and weighted F1 metric. This is because EW tends to concentrate the nodes with similar features (labels) to a single partition. Although this may cause a slight degradation in convergence during the generalization phase, it improves the micro-F1 scores during the personalization phase as each model adapts to its own local distribution (Fig 3). GP also significantly reduces the communication overhead of averaging gradients across all partitions, as shown by the good speed-up achieved on large graph benchmarks. CBS helps alleviate the imbalance in class distribution caused by EW partitioning and helps speed up the convergence significantly (2x-3x) for large graph benchmarks. We also note that GP+CBS can also speed up (1.75x on average) **DistDGL** with METIS partition while maintaining the same accuracy. We thus conclude that **EW+GP+CBS** should be used to maximize the micro-F1 score (accuracy) and minimize training time while distributed training of large graph benchmarks.

Table V shows the pre-processing time for different partitioning schemes. **EW** incurs additional overhead due to the time spent by the PyMETIS library in performing edge-weighted partitioning. In Yelp, about 23.2% of the time is spent on edge weight assignment, while 76.7% of the total time is spent on PyMETIS calls.

VI. Conclusion

This paper presents the importance of entropy-aware training algorithms to improve the performance of distributed GNN. The edge-weighted partitioning technique proves to be effective for improving Micro-F1. The training techniques: personalization and class-balanced sampling help simultaneously improve performance metrics and reduce training time. Through extensive experimental evaluation on the DistDGL framework, we achieved a (2-3x) speedup in training time and 4% improvement on average in micro-F1 scores of 5 large graph benchmarks compared to the standard baselines.

Acknowledgment

The support and the resources provided by PARAM Shakti at the Indian Institute of Technology, Kharagpur and PARAM Sanganak at the Indian Institute of Technology, Kanpur under the National Supercomputing Mission, Government of India are gratefully acknowledged. Vishwesh Jatala acknowledges the funding received from DST/SERB through Start-up Research Grant SRG/2021/001134.

References

[1] K. e. a. Duan, "A comprehensive study on large-scale graph training: Benchmarking and rethinking," in *Thirty-sixth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2022.

[2] Y. e. a. Liu, "Pick and choose: A gnn-based imbalanced learning approach for fraud detection." Association for Computing Machinery, 2021.

[3] D. Zheng, C. Ma, M. Wang, J. Zhou, Q. Su, X. Song, Q. Gan, Z. Zhang, and G. Karypis, "Distdgl: Distributed graph neural network training for billion-scale graphs," in *2020 IEEE/ACM 10th Workshop on Irregular Applications: Architectures and Algorithms (IA3)*, 2020, pp. 36–44.

[4] S. Gandhi and A. P. Iyer, "P3: Distributed deep graph learning at scale," in *OSDI*. USENIX Association, Jul. 2021, pp. 551–568.

[5] W.-L. Chiang, X. Liu, S. Si, Y. Li, S. Bengio, and C.-J. Hsieh, "Cluster-gcn: An efficient algorithm for training deep and large graph convolutional networks." New York, NY: Association for Computing Machinery, 2019.

[6] T. Li, S. Hu, A. Beirami, and V. Smith, "Ditto: Fair and robust federated learning through personalization," PMLR, pp. 6357–6368, 2021.

[7] Z. Luo, J. Lian, H. Huang, H. Jin, and X. Xie, "Ada-gnn: Adapting to local patterns for improving graph neural networks," February 2022.

[8] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," Red Hook, NY, USA, p. 1025–1035, 2017.

[9] K. D. et. el., "A comprehensive study on large-scale graph training: Benchmarking and rethinking," in *Thirty-sixth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2022.

[10] J. You, R. Ying, and J. Leskovec, "Design space for graph neural networks," ser. NIPS'20. Curran Associates Inc., 2020.

[11] C. Wan, Y. Li, C. R. Wolfe, A. Kyrillidis, N. S. Kim, and Y. Lin, "PipeGCN: Efficient full-graph training of graph convolutional networks with pipelined feature communication," in *The Tenth International Conference on Learning Representations (ICLR 2022)*, 2022.

[12] U. Iqbal, Z. Shafiq, P. Snyder, S. Zhu, Z. Qian, and B. Livshits, "Adgraph: A machine learning approach to automatic and effective adblocking," *CoRR*, 2018.

[13] V. M. et. al., "Distgnn: Scalable distributed training for large-scale graph neural networks," *CoRR*, vol. abs/2104.06700, 2021.

[14] A. Abou-Rjeili and G. Karypis, "Multilevel algorithms for partitioning power-law graphs," in *IPDPS*.

[15] I. Stanton and G. Kliot, "Streaming graph partitioning for large distributed graphs." New York, NY, USA: Association for Computing Machinery, 2012.

[16] G. Karypis and V. Kumar, "A fast and high quality multilevel scheme for partitioning irregular graphs," *SIAM J. Sci. Comput.*, dec 1998.

[17] R. Z. et. al., "Aligraph: A comprehensive graph neural network platform," *CoRR*, 2019.

[18] T. Zhao, X. Zhang, and S. Wang, "Graphsmote: Imbalanced node classification on graphs with graph neural networks." Association for Computing Machinery, 2021.

[19] H. Zeng, H. Zhou, A. Srivastava, R. Kannan, and V. K. Prasanna, "Graphsaint: Graph sampling based inductive learning method," 2020.

[20] X. Tang, S. Guo, and J. Guo, "Personalized federated learning with contextualized generalization," in *Proceedings of the Thirty-First IJCAI*, L. D. Raedt, Ed., 2022.

[21] D. H. et. al., "Splitgp: Achieving both generalization and personalization in federated learning," *CoRR*, 2022.

[22] M. L. Das, V. Jatala, and G. R. Gupta, "Joint partitioning and sampling algorithm for scaling graph neural network," in *2022 IEEE 29th International Conference on High Performance Computing, Data, and Analytics (HiPC)*, 2022, pp. 42–47.

[23] M. Wang and L. Y. et. al., "Deep graph library: Towards efficient and scalable deep learning on graphs," *CoRR*, 2019.